



TC2038 - Análisis y diseño de algoritmos avanzados

Tema 2.1 Algoritmo Manacher - Palíndromo más Largo

Profesor - Alison Muñoz Capote
Computación - Facultad de Ingeniería

Mapa del Tema y Objetivos

Objetivos de la Sesión:

1. **Comprender** las limitaciones geométricas y temporales de la búsqueda tradicional de palíndromos en cadenas de texto.
2. **Asimilar** el concepto de simetría y reutilización de memoria para optimizar algoritmos de $O(N^2)$ a $O(N)$.
3. **Analizar e implementar** el algoritmo de Manacher mediante la transformación de cadenas y el cálculo del arreglo de radios.

Índice:

1. Introducción.
2. Los 4 Pilares de Manacher (Delimitadores, Arreglo P, Centro/Borde, Espejo).
3. Caso de Estudio Paso a Paso (`abacaba`).
4. Diseño e Implementación.
5. Problemas y Trabajo Independiente.

1. Resumen LPS, KMP y Función Z

El Arreglo LPS (Función Pi)

Objetivo: Preprocesar el patrón para evitar comparaciones redundantes.

Definición: `pi[i]` o `lps[i]` almacena la longitud del prefijo propio más largo que también es sufijo en la subcadena `patron[0...i]`.

La regla de oro (El salto `j = pi[j-1]`):

Cuando `patron[i] != patron[j]`, no regresamos `j` a 0 inmediatamente. Consultamos el arreglo `pi` para retroceder al siguiente prefijo más largo disponible dentro de la *zona segura* que ya habíamos coincidido.

LPS: Ejemplo representativo (ABACABAB)

Calculamos `pi` para el patrón `A B A C A B A B`.

Hasta `i = 6`, venimos construyendo una racha perfecta de tamaño 3 (`A B A`):

i (Índice)	0	1	2	3	4	5	6	7
Caracter	A	B	A	C	A	B	A	B
pi[i]	0	0	1	0	1	2	3	?

Al llegar a `i = 7` (Caracter `B`), nuestra variable `j` vale 3 (porque `pi[6] = 3`).

LPS: Desglose del rescate en $i=7$

Aquí ocurre la magia de KMP. El patrón intenta crecer a tamaño 4, falla, pero logra reciclar información para no caer hasta 0.

1. Intento 1 ($j=3$):

- Esperamos `patron[3]` que es 'C'.
- Tenemos `patron[7]` que es 'B'.
- **Desajuste** ('C' != 'B'). Entramos al `while`.

2. El salto inteligente ($j = pi[j-1]$):

- Consultamos el prefijo más largo dentro de nuestra zona segura (`A B A`).
- Salto: `j = pi[3-1] = pi[2]`.
- Miramos la tabla: `pi[2]` vale **1**. Ahora `j = 1`.

LPS: La coincidencia salvadora

3. Intento 2 (j=1 con el nuevo valor):

- El ciclo `while` vuelve a evaluar con nuestro nuevo `j`.
- Esperamos `patron[1]` que es 'B'.
- Tenemos `patron[7]` que es 'B'.
- **iCoincidencia!** ('B' == 'B').

4. Conclusión del ciclo:

- El `while` se rompe inmediatamente.
- Sumamos 1 a nuestra racha rescatada: `j = j + 1` (es decir, `j = 2`).
- Guardamos el resultado: `pi[7] = 2`.

Resultado final: El prefijo-sufijo más largo hasta el índice 7 es **A B**.

El Algoritmo KMP (Knuth-Morris-Pratt)

Objetivo: Buscar las ocurrencias de un patrón dentro de un texto en tiempo $O(n + m)$.

Lógica:

Utiliza el arreglo LPS precalculado. Cuando hay un desajuste comparando el texto con el patrón, KMP usa el LPS para saber exactamente cuántos caracteres del patrón ya están *alineados* implícitamente con el texto, saltándose las posiciones que matemáticamente no pueden coincidir.

KMP: Ejemplo simple

- **Texto:** A B A B X A B A B C
- **Patrón:** A B A B C (Su LPS es $[0, 0, 1, 2, 0]$)

Ejecución:

1. Comparamos hasta el índice 4: Texto tiene X, Patrón espera C. (Desajuste).
2. Caracteres coincidentes hasta ahora: $j = 4$ (A B A B).
3. KMP consulta: $j = \text{LPS}[4-1] = \text{LPS}[3] = 2$.
4. El patrón se *desliza* alineando su prefijo de tamaño 2 (A B) con el sufijo que acabamos de leer en el texto.
5. Continuamos comparando desde el índice 2 del patrón contra la X. Como $X \neq A$, vuelve a saltar: $j = \text{LPS}[2-1] = 0$.
6. Avanzamos en el texto y encontramos la coincidencia completa más adelante.

3. El Algoritmo Z (Función Z)

Objetivo: Encontrar todas las ocurrencias de un patrón en un texto en tiempo $O(n + m)$ usando una caja de coincidencia (Z-box).

Definición: $Z[i]$ almacena la longitud de la subcadena más larga que comienza en $S[i]$ y que simultáneamente es un prefijo de la cadena completa S .

Uso clásico para búsqueda:

Se concatena el patrón, un carácter separador (ej. $\$$), y el texto: $Patron\$Texto$. Si en la posición i del texto resulta que $Z[i] == longitud(Patron)$, hay una coincidencia exacta.

Función Z: Ejemplo sencillo

Calculamos el arreglo Z para la cadena: **A B A C A B A**

i (Índice)	0	1	2	3	4	5	6
Caracter	A	B	A	C	A	B	A
Z[i]	-	0	1	0	3	0	1

- **Z[2] = 1**: La subcadena inicia con **A**, que coincide con el prefijo de tamaño 1 del string original.
- **Z[4] = 3**: La subcadena inicia con **A B A**, que coincide perfectamente con el prefijo inicial de tamaño 3 del string original (**A B A**).

Algoritmo de Manacher

Situación Problema: La Motivación

Imagina que trabajas en **Bioinformática** analizando secuencias de ADN.

Buscas secuencias palindrómicas, ya que ciertas enzimas de restricción cortan el ADN en sitios específicos que son palíndromos (ej. `GAATTC`).

El genoma humano tiene miles de millones de pares de bases. Un algoritmo ineficiente tardaría años en encontrar las secuencias más largas.

Definición del Problema

Entrada: Una cadena de texto `s` de longitud `N`.

Salida: La subcadena contigua más larga dentro de `s` que se lee igual de izquierda a derecha que de derecha a izquierda (un palíndromo).

Ejemplo:

Para `s = "babad"`, las respuestas válidas son `"bab"` o `"aba"`.

Para `s = "cbbd"`, la respuesta válida es `"bb"`.

El Enfoque Ingenuo (Fuerza Bruta)

La solución más obvia:

1. **Generar** todas las subcadenas posibles.
2. **Verificar** si cada una es un palíndromo.
3. **Guardar** la más larga.

Complejidad:

- Generar subcadenas: $O(N^2)$
- Verificar cada una: $O(N)$
- **Total:** $O(N^3)$

Inviabile para cadenas de ADN o textos medianamente largos.

Mejora: Expansión desde el Centro

Todo palíndromo tiene un centro. Podemos iterar por cada carácter del texto y expandirnos hacia la izquierda y derecha mientras los caracteres coincidan.

Problema 1: El tiempo.

En el peor caso (ej. "aaaaaa"), el algoritmo expande repetidamente caracteres que ya fueron evaluados.

Total: $O(N^2)$

Problema 2 de la Expansión: La Paridad

Los palíndromos tienen dos formas:

- **Impares:** Tienen un único centro real (ej. `a b a` -> centro `b`).
- **Pares:** El centro está *entre* dos caracteres (ej. `a b b a` -> centro entre `b` y `b`).

Para programar la expansión, hay que hacer dos bucles en cada posición: uno asumiendo centro impar y otro asumiendo centro par.

La Solución Elegante: Glenn Manacher (1975)

Manacher propuso un algoritmo que resuelve ambos problemas (la redundancia $O(N^2)$ y la paridad par/impar) en un solo pase lineal.

Complejidad: $O(N)$ en tiempo y $O(N)$ en espacio.

A continuación, veremos los 4 pilares conceptuales que hacen posible esta magia.

Concepto 1: Preprocesamiento (Delimitadores)

Para unificar los palíndromos pares e impares, inyectamos un carácter especial (ej. #) entre cada letra, al principio y al final.

- aba (impar) -> # a # b # a #
- abba (par) -> # a # b # b # a #

¡Ahora **todos** los palíndromos en la nueva cadena tienen una longitud impar, con un centro exacto!

Concepto 1: Fronteras de Seguridad

Para evitar comprobaciones de *fuera de límites* (Index Out of Bounds) en los extremos del arreglo al expandir, añadimos dos caracteres únicos distintos al principio y al final (ej. `^` y `$`).

String original: `babad`

String procesado: `^ # b # a # b # a # d # $`

Concepto 2: El Arreglo de Radios (P)

Creamos un arreglo `P` del mismo tamaño que nuestra nueva cadena.

En cada índice `i`, `P[i]` almacenará el **radio** del palíndromo más largo centrado en `i`.

El radio no cuenta el centro. Si `P[i] = 3`, significa que el palíndromo se expande 3 posiciones a la izquierda y 3 a la derecha.

Concepto 3: Centro (C) y Borde Derecho (R)

El secreto de $O(N)$ es no recalcular lo ya calculado.

Para ello, el algoritmo memoriza el palíndromo que ha logrado expandirse más hacia la derecha en toda la ejecución.

- **C (Center):** El índice central de ese palíndromo ganador.
- **R (Right):** El límite derecho absoluto de ese palíndromo ($R = C + P[C]$).

Concepto 4: El Espejo (i')

Si estamos evaluando una posición i que está **dentro** de nuestro límite derecho R ($i < R$), ino empezamos de cero!

Al estar dentro de un palíndromo mayor (centrado en C), la posición i tiene un "espejo" en la mitad izquierda del palíndromo, llamado i' .

Fórmula: $i' = 2 * C - i$

La Regla de Oro de Manacher

Si $i < R$, el radio en i será, al menos, igual al radio de su espejo i' .
¡Simetría pura!

La única restricción:

El radio heredado del espejo no puede forzar a nuestro nuevo palíndromo a salirse más allá del borde R que ya conocemos.

Por tanto: $P[i] = \min(R - i, P[i'])$

Caso de Estudio: Preparación

Buscaremos el palíndromo más largo en la cadena `S = "abacaba"`.

Paso 1: Preprocesamiento

Transformamos `S` a `T`.

```
T = [^, #, a, #, b, #, a, #, c, #, a, #, b, #, a, #, $]
```

Longitud original: 7. Nueva longitud: 17.

Caso de Estudio: Inicialización

Preparamos nuestras variables:

`P` = Arreglo de 17 ceros.

`C = 0` (Centro actual)

`R = 0` (Borde derecho actual)

Iniciamos el bucle desde `i = 1` hasta `15`.

Caso de Estudio: Iteraciones Iniciales

Para los primeros índices (ej. `#`, `a`, `#`), `i >= R`, así que no podemos usar el espejo. Expandimos manualmente comparando izquierda y derecha.

Cuando procesamos la primera subcadena `aba`:

En `i = 4` (la primera letra `b` en `#a#b#a#`):

El palíndromo se expande radio 3.

`P[4] = 3`

Actualizamos: `C = 4`, `R = 4 + 3 = 7`.

Caso de Estudio: El Gran Palíndromo

Avanzamos hasta $i = 8$ (la letra central c).

$i(8) \geq R(7)$, no hay espejo válido. Expandimos a fuerza bruta.

La expansión tiene éxito completo. Todo el texto $\#a\#b\#a\#c\#a\#b\#a\#$ es un palíndromo.

$P[8] = 7$

Actualizamos récord: $C = 8$, $R = 8 + 7 = 15$.

Caso de Estudio: La Magia del Espejo

Avanzamos a `i = 12` (la segunda letra `b`).

¡Ojo! Nuestro borde `R` es `15`. Como `12 < 15`, **podemos usar el espejo**.

Cálculo del espejo de 12 respecto al centro 8:

$$i' = 2 * 8 - 12 = 4$$

Miramos qué pasó en el espejo: `P[4]` era `3`.

Caso de Estudio: Reutilizando Trabajo

Sabemos que el radio mínimo garantizado es:

```
P[12] = min(R - i, P[i'])
```

```
P[12] = min(15 - 12, P[4])
```

```
P[12] = min(3, 3) = 3
```

Le damos a `P[12]` un valor base de 3 sin hacer *ninguna* comparación de caracteres.

Caso de Estudio: Intento de Expansión

A partir del radio base 3, intentamos expandir en $i = 12$:

Comparamos $T[12 + 3 + 1]$ con $T[12 - 3 - 1]$

Comparamos $T[16]$ con $T[8]$ -> s contra c .

¡No coinciden! La expansión se detiene inmediatamente.

El bucle continúa. Nos hemos ahorrado todo el trabajo central.

Resultado del Caso de Estudio

El arreglo `P` final para `^#a#b#a#c#a#b#a#$` se ve así:

Caracter	^	#	a	#	b	#	a	#	c	#	a	#	b	#	a	#	\$
Índice <code>i</code>	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
<code>P[i]</code>	0	0	1	0	3	0	1	0	7	0	1	0	3	0	1	0	0

El número mayor es **7** en el índice **8**.

Recuperando el String Original

El palíndromo más largo tiene longitud `7`.

El centro en la cadena modificada es `i = 8`.

Para encontrar el inicio en la cadena original, usamos una fórmula mágica:

```
Inicio = (Centro - Radio) / 2
```

```
Inicio = (8 - 7) / 2 = 0
```

El palíndromo empieza en el índice 0 y mide 7 caracteres: `"abacaba"`.

Análisis de Complejidad Temporal $O(N)$

Aunque hay un bucle `while` anidado dentro de un `for`, la complejidad es estrictamente $O(N)$. ¿Por qué?

El bucle `while` solo se ejecuta con éxito cuando expande el palíndromo más allá del límite derecho actual `R`.

La variable `R` empieza en 0 y solo puede crecer hasta `2N + 1`.

Por tanto, el `while` anidado avanzará un máximo total de `2N` veces a lo largo de *toda* la ejecución del programa.

Análisis de Complejidad Espacial $O(N)$

El algoritmo requiere memoria extra, pero crece de forma lineal respecto a la entrada.

1. **Cadena transformada (T):** Su tamaño es $2N + 3$.
2. **Arreglo de radios (P):** Su tamaño es $2N + 3$.

Espacio total: $O(N)$ auxiliar, lo cual es excelente y manejable incluso para textos muy largos.

Casos Especiales y Extremos

1. Cadena de 1 carácter ("a"):

Se transforma a `^#a#$`. `P` halla un radio de 1 en la 'a'.
Retorna longitud 1. (Correcto).

2. Cadena vacía (""):

Casi nunca es entrada válida, pero preprocesar retorna `^$`. El bucle principal no se ejecuta por los límites, y la respuesta es vacía. (Seguro).

3. Todos los caracteres iguales ("aaaa"):

Se transforma a `^#a#a#a#a#$`.

El algoritmo detecta eficientemente centros pares e impares, empujando `R` hacia adelante de golpe. Gracias al espejo, las iteraciones siguientes solo confirman el límite sin re-evaluar todo el centro repetido $O(N)$ en lugar de $O(N^2)$.

Diseño del Algoritmo (Flujo)

1. Construir la cadena preprocesada `T`.
2. Crear arreglo `P` lleno de ceros.
3. Inicializar `C = 0`, `R = 0`.
4. Bucle `for i` desde 1 hasta tamaño de `T - 1`:
 - Calcular `espejo`.
 - Si `i < R`, inicializar `P[i]`.
 - Bucle `while`: Expandir desde `i`.
 - Si pasamos de `R`, actualizar `C` y `R`.
 - Rastrear el índice con el mayor valor en `P`.
5. Extraer y retornar la subcadena original.

Pseudocódigo (Preprocesamiento)

```
Función Preprocesar(S):  
    T = "^"  
    Para cada caracter c en S:  
        T = T + "#" + c  
    T = T + "$"  
    Retornar T
```

Las barreras `^` y `$` previenen desbordamientos durante el ciclo while posterior.

Pseudocódigo (Lógica Central)

```
C = 0, R = 0
centro_max = 0, max_long = 0
Para i = 1 hasta longitud(T) - 2:
    espejo = 2*C - i
    Si i < R:
        P[i] = min(R - i, P[espejo])
    Mientras T[i + 1 + P[i]] == T[i - 1 - P[i]]:
        P[i]++
    Si i + P[i] > R:
        C = i
        R = i + P[i]
    Si P[i] > max_long:
        max_long = P[i]
        centro_max = i
```

Implementación de la Transformación

```
def algoritmo_manacher(s: str) -> str:
    if not s:
        return ""

    # 1. Preprocesamiento
    # 'aba' -> '^#a#b#a#$'
    T = ['^']
    for char in s:
        T.append('#')
        T.append(char)
    T.append('#')
    T.append('$')

    n = len(T)
    P = [0] * n
```


Definir Variables

```
# Variables para rastrear el palíndromo más a la derecha  
C = 0  # Centro  
R = 0  # Límite derecho  
  
# Variables para rastrear el palíndromo máximo encontrado  
max_longitud = 0  
centro_max = 0
```

Implementación del Bucle Principal

```
for i in range(1, n - 1):
    espejo = 2 * C - i

    # Si estamos dentro de R, aprovechamos el espejo
    if i < R:
        P[i] = min(R - i, P[espejo])

    # Intentar expandir el palíndromo centrado en i
    while T[i + 1 + P[i]] == T[i - 1 - P[i]]:
        P[i] += 1

    # Si nos expandimos más allá de R, actualizamos C y R
    if i + P[i] > R:
        C = i
        R = i + P[i]
```

Extracción Final

```
# Rastrear el palíndromo más largo global
    if P[i] > max_longitud:
        max_longitud = P[i]
        centro_max = i

# Extraer de la cadena original
# Fórmula: Inicio = (Centro - Radio) // 2
inicio = (centro_max - max_longitud) // 2
fin = inicio + max_longitud

return s[inicio:fin]
```

Reto 1: Conteo Total de Palíndromos

Descripción: Dada una cadena de texto `s`, devuelve el *número total* de subcadenas palindrómicas continuas que contiene. (Ejemplo: `"aaa"` tiene 6: `"a"`, `"a"`, `"a"`, `"aa"`, `"aa"`, `"aaa"`).

Reto: Resolverlo en tiempo $O(N)$.

Hints para la solución:

- Ejecuta la construcción del arreglo `P` (radios) del algoritmo de Manacher tal cual lo vimos.
- En la cadena transformada, cada valor `P[i]` no solo te da el palíndromo más largo, sino que **dicta cuántos palíndromos** comparten ese mismo centro.
- Matemáticamente, el número de palíndromos centrados en la posición `i` de la cadena transformada es exactamente `(P[i] + 1) // 2`. Solo debes iterar `P` y sumar estos valores.

Reto 2: El Palíndromo Más Corto

Descripción: Dada una cadena `s`, puedes convertirla en un palíndromo añadiendo caracteres *únicamente al principio*. Encuentra el palíndromo más corto que puedes formar. (Ejemplo: `s = "abcd"` $\rightarrow$ `"dcbabcd"`, `s = "aacecaaa"` $\rightarrow$ `"aaacecaaa"`).

Reto: Resolverlo en tiempo $O(N)$.

Hints para la solución:

- El problema equivale a encontrar el **prefijo palindrómico más largo** de la cadena original `s`. Los caracteres que no formen parte de ese prefijo son los que debes invertir y pegar al principio.
- Ejecuta Manacher. Busca el índice `i` en la cadena transformada tal que su palíndromo alcance el borde izquierdo extremo (es decir, `i - P[i] == 0` o el índice correspondiente al inicio lógico).
- El `P[i]` más grande que cumpla esa condición te dirá qué parte de `s` ya es un palíndromo desde el índice 0.

Conclusión

El **Algoritmo de Manacher** es un ejemplo de cómo el análisis de propiedades matemáticas (la simetría en palíndromos) puede transformar un problema aparentemente pesado y redundante $O(N^2)$ en una solución lineal elegante $O(N)$.

Su preprocesamiento inteligente y el uso de memoria para saltar cálculos lo convierten en la solución definitiva para este *problema clásico*.

Estudio Independiente

Trabajo Independiente 1: Longest Common Substring

Problema de Tarea: Dadas dos cadenas de texto `s1` y `s2`, encontrar la longitud de la subcadena continua más larga que aparece en ambas. (Ejemplo: `s1 = "ABABC"`, `s2 = "BABCA"` → Res: 4, `"BABC"`).

Orientación de Lecturas y Conceptos:

- **Programación Dinámica (DP):** Investiga cómo construir una matriz `LCS[i][j]` donde almacenes la longitud del sufijo común más largo para los prefijos `s1[0..i]` y `s2[0..j]`. Tiempo: $O(N \times M)$.
- **Autómata de Sufijos (Suffix Automaton):** Un grafo acíclico dirigido que reconoce todos los sufijos de una cadena. Permite resolver este problema en tiempo lineal $O(N + M)$.
- **Árbol de Sufijos Generalizado (Generalized Suffix Tree):** Concepto teórico clave para concatenar `s1 + '#' + s2` y buscar el nodo interno más profundo con hojas de ambas cadenas originales.

Trabajo Independiente 2: Suffix Array y Hashing

Problema de Tarea: Dada una cadena de texto enorme, encuentra la **subcadena repetida más larga** que aparece al menos dos veces sin solaparse. (Ejemplo: "banana" → "an" o "na").

Orientación de Lecturas y Conceptos:

- **Arreglo de Sufijos (Suffix Array):** Un arreglo ordenado alfabéticamente de todos los sufijos de una cadena. Estudia el algoritmo de construcción en $O(N \log N)$ (Ej. algoritmo de prefijos duplicados).
- **Arreglo LCP (Longest Common Prefix):** Se construye a la par del Suffix Array (Algoritmo de Kasai). El valor máximo en el arreglo LCP suele dar la longitud de la subcadena repetida más larga.
- **Polynomial Rolling Hash (Rabin-Karp):** Investiga cómo calcular el hash de subcadenas en tiempo $O(1)$ tras un preprocesamiento. Útil para hacer búsqueda binaria sobre la longitud de la posible subcadena repetida.